WEEK 8: PHASE 3
Gold Layer & Spark Tuning

- ✥ Materialized Views in Lakeflow (@dp.materialized_view — corrected)
- ✥ Streaming Tables & AUTO CDC ... INTO (replaces APPLY CHANGES)
- ✥ DLT Expectations (expect, expect_or_drop, expect_or_fail)
- ✥ Spark tuning: shuffle.partitions, autoBroadcastJoinThreshold (10MB vs 30MB AQE)
- ✥ OPTIMIZE, ZORDER, Liquid Clustering
- ✥ 10 Practice Questions + Full Answer Key

Table of Contents

1. Gold Layer Object Types 3
2. Materialized Views in Lakeflow [CORRECTED API] 4
3. Streaming Tables & AUTO CDC [NEW SYNTAX] 5
4. DLT Data Quality Expectations 7
5. Spark Tuning Parameters [EXPANDED] 9
6. OPTIMIZE, ZORDER & Liquid Clustering 11
7. Phase 3 Summary & Key Facts 13
8. Practice Questions (10 Qs) 14
9. Answer Key 17

GOLD LAYER

1. Gold Layer Object Types

MV · Views · Streaming · Tables

The Gold layer serves business intelligence and analytics. Databricks provides four types of objects for Gold layer output, each with different trade-offs in latency, freshness, and compute cost.

Object Type	Storage	Refresh	Latency	Best For
Delta Table	Physical	Manual/scheduled	Batch	Final ETL output
SQL View	None — virtual	On every query	Query-time	Simple transforms
Materialized View	Physical	Auto by pipeline	Near-real-time	Dashboards
Streaming Table	Physical	Continuous	Low latency	CDC feeds

■ **Exam Focus:** Views have NO storage — they re-compute on every query. Materialized Views have physical storage and are auto-refreshed. Regular tables require explicit writes.

SECTION 2 **2. Materialized Views in Lakeflow** API Corrected: @dp.materialized_view

■ **CRITICAL CORRECTION (July 2026):** The @dlt module and @dlt.table decorator for Materialized Views are now deprecated. Use @dp (pyspark.pipelines) with @dp.materialized_view.

Materialized View Creation (CORRECTED)

■ ✔■ **Correct: Lakeflow MV with @dp.materialized_view**

```
# OLD (DEPRECATED) — DO NOT USE
import dlt
@dlt.table(name='daily_revenue_mv')
def daily_revenue_mv():
    return spark.sql('SELECT ...')

# NEW (CORRECT) — Use this
from pyspark import pipelines as dp

@dp.materialized_view(
    name='daily_revenue_mv',
    comment='Gold — daily revenue by region'
)
def daily_revenue_mv():
    return spark.sql('''
        SELECT order_date, region,
        COUNT(DISTINCT customer_id),
        SUM(amount) AS total_revenue
        FROM LIVE.silver_orders
        GROUP BY order_date, region
    ''')
```

Materialized View vs SQL View

Feature	Materialized View	SQL View
Storage	Physical Delta files	No — virtual only
Performance	Fast (pre-computed)	Slow (re-computes)
Refresh	Auto by pipeline	N/A — always re-computes
Expectations	Yes (Lakeflow)	No

Reference: https://docs.databricks.com/aws/en/ldp/concepts/where-is-dlt (July 2026)

SECTION 3 **3. Streaming Tables & AUTO CDC** APPLY CHANGES → AUTO CDC (New Syntax)

■ **CRITICAL CORRECTION (July 2026):** APPLY CHANGES INTO is being replaced by the new AUTO CDC ... INTO syntax. While legacy syntax still works, AUTO CDC is the recommended approach.

Streaming Table with Auto Loader

■ ✔■ **Python: @dp.table for Streaming Table**

```
from pyspark import pipelines as dp
from pyspark.sql.functions import current_timestamp

@dp.table(name='bronze_events')
def bronze_events():
    return (
        spark.readStream
        .format('cloudFiles')
        .option('cloudFiles.format', 'json')
        .option('cloudFiles.schemaLocation',
            '/Volumes/cat/sch/vol/schema/events')
        .load('s3://bucket/raw/events/')
        .withColumn('_ingest_ts', current_timestamp())
    )
```

Change Data Capture (CDC) — OLD vs NEW

OLD (APPLY CHANGES INTO) — DEPRECATED:

■ **Deprecated (still works)**

```
APPLY CHANGES INTO silver_customers
FROM STREAM(bronze_customers_cdc)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY change_ts
COLUMNS * EXCEPT (operation, change_ts);
```

NEW (AUTO CDC ... INTO) — RECOMMENDED:

■ ✔■ **New (Recommended)**

```
CREATE OR REFRESH STREAMING TABLE silver_customers AS
AUTO CDC FLOW INTO silver_customers
FROM STREAM(LIVE.bronze_customers_cdc)
KEYS (customer_id)
SEQUENCE BY change_ts
COLUMNS * EXCEPT (operation, change_ts);
```

AUTO CDC automatically handles out-of-sequence records. Python API: create_auto_cdc_flow() (replaces apply_changes()).

SECTION 4

4. DLT Data Quality Expectations

expect · expect_or_drop · expect_or_fail

DLT Expectations define data quality rules at the table level. They run on every batch and either track, drop, or fail based on the decorator used.

Decorator	Behavior	Row Kept?	Pipeline Fails?
@dp.expect()	Record violation metric	Yes	No
@dp.expect_or_drop()	Drop violating rows	No	No
@dp.expect_or_fail()	Fail pipeline	No	Yes

Python Expectations

■ ✔■ **Three expectation decorators**

```
from pyspark import pipelines as dp

@dp.table(name='silver_orders')
@dp.expect('valid_order_id', 'order_id IS NOT NULL')
@dp.expect_or_drop('positive_amount', 'amount > 0')
@dp.expect_or_fail('valid_status',
    'status IN ("PENDING", "ACTIVE", "COMPLETE")')
def silver_orders():
    return spark.table('LIVE.bronze_orders')
```

SQL Expectations

■ ✔■ **ON VIOLATION DROP ROW / FAIL UPDATE**

```
CREATE OR REFRESH STREAMING TABLE silver_orders
CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL)
CONSTRAINT positive_amount EXPECT (amount > 0)
  ON VIOLATION DROP ROW
CONSTRAINT valid_status EXPECT (status IN
  ('PENDING','ACTIVE','COMPLETE'))
  ON VIOLATION FAIL UPDATE
AS SELECT * FROM STREAM(LIVE.bronze_orders);
```

expect_or_fail causes the entire pipeline update to fail atomically. Use only for critical validations.

SECTION 5

5. Spark Tuning Parameters

Detailed: shuffle.partitions & autoBroadcastJoinThreshold

Understanding key Spark configuration parameters is essential for fixing performance issues.

Parameter	Default	Purpose
spark.sql.shuffle.partitions	200	Partitions after shuffle/join/agg
spark.sql.autoBroadcastJoinThreshold	10MB	Static threshold for auto-broadcast
spark.databricks.adaptive.autoBroadcastJoinThreshold	30MB	AQE runtime threshold (Databricks)
spark.sql.adaptive.enabled	true	Adaptive Query Execution (AQE)

■ **Databricks Default (June 2026):** AQE is enabled by default. The AQE runtime threshold is 30MB, not static 10MB. Most manual tuning is no longer necessary.

shuffle.partitions Tuning

■ **shuffle.partitions tuning**

```
# 500MB dataset into 200 partitions by default:
# Most tasks finish in <1s with many empty partitions
# Rule of thumb: 128MB–256MB per partition

# For 500MB data: 500MB / 256MB ≈ 2 partitions
spark.conf.set('spark.sql.shuffle.partitions', '4')

# AQE enabled (default) auto-coalesces small partitions
```

autoBroadcastJoinThreshold — TWO Settings

Static (10MB): Used for non-AQE queries. Tables <10MB are auto-broadcast.

AQE Runtime (30MB): If one side shrinks <30MB after filtering, Spark converts to broadcast join at runtime.

■ **autoBroadcastJoinThreshold examples**

```
# 50MB dimension table, 5TB fact table
# Static 10MB says: No auto-broadcast
# AQE 30MB says: If filtered <30MB, broadcast it

# Manual override (for stable <200MB dims):
spark.conf.set('spark.sql.autoBroadcastJoinThreshold',
               200*1024*1024)  # 200MB
```

■ **Exam Note:** Know both values: 10MB static, 30MB AQE runtime. Most Databricks uses AQE, so 30MB matters.

SECTION 6 **6. OPTIMIZE, ZORDER & Liquid Clustering** File compaction · Data skipping

OPTIMIZE — File Compaction

■ **OPTIMIZE for file compaction**

```
OPTIMIZE catalog.silver.orders;
DESCRIBE DETAIL catalog.silver.orders;
-- Look for: numFiles, sizeInBytes, averageFileSize
```

ZORDER — Legacy Clustering

■ **ZORDER (legacy)**

```
OPTIMIZE catalog.silver.orders
ZORDER BY (customer_id, order_date);
-- High cost: rewrites all files for co-location
```

Liquid Clustering — Modern (Recommended)

■ **✔■ Liquid Clustering (recommended)**

```
CREATE TABLE catalog.silver.orders
CLUSTER BY (customer_id, order_date)
AS SELECT * FROM catalog.bronze.orders;

OPTIMIZE catalog.silver.orders;  -- incremental
```

Feature	ZORDER	Liquid Clustering
Cost	Full re-write	Incremental (only changed)
Recommendation	Legacy tables	All new tables (DBR 15.4+)

■ Recommendation: Always use Liquid Clustering for new tables (DBR 15.4 LTS+). Lower cost, same performance.

SUMMARY	7. Phase 3 Summary	Weeks 6–8: Transformation & Modeling (22%)

Critical Corrections (Week 8)

✓ Materialized View API: @dlt.table → @dp.materialized_view

✓ CDC Syntax: APPLY CHANGES INTO → AUTO CDC ... INTO

✓ autoBroadcastJoinThreshold: 10MB (static) vs 30MB (AQE runtime)

✓ Liquid Clustering: Recommended for all new tables

Week-by-Week Breakdown

Week	Key Topics
6	Null handling, Type casting, Column ops, MERGE INTO
7	Join types, Broadcast, Deduplication, Aggregations
8	Gold objects, Expectations, Spark tuning, OPTIMIZE/ZORDER/Liquid Clustering

Practice Questions (Week 8)

1. You create a Materialized View using @dlt.table. You get an AttributeError. Issue?

A) @dlt.table is deprecated; use @dp.materialized_view from pyspark.pipelines

B) The table name must be lowercase

C) You must import dlt before using @dlt

D) Materialized Views require SQL syntax, not Python

2. CDC feed with out-of-order updates. APPLY CHANGES works but loses data. Recommended modern approach?

A) Use MERGE INTO instead

B) Switch to AUTO CDC ... INTO (handles out-of-order automatically)

C) Increase spark.sql.shuffle.partitions

D) Use a Streaming Table without CDC logic

3. @dp.expect_or_fail("critical_id", "id IS NOT NULL"). Row with id=null arrives. What happens?

A) Row is kept; violation is logged in metrics

B) Row is silently dropped

C) Pipeline update fails and rolls back atomically

D) Row is moved to a quarantine table

4. 500MB dataset into 200 partitions by default. Most tasks <1s with empty partitions. Fix?

A) Increase spark.sql.shuffle.partitions to 1000

B) Decrease to ~2-4; AQE will auto-coalesce

C) Set autoBroadcastJoinThreshold to 500MB

D) Run OPTIMIZE to compact files

5. 50MB dimension table. Static threshold is 10MB. What will broadcast it at runtime?

A) Set spark.sql.autoBroadcastJoinThreshold to 100MB

B) Set spark.databricks.adaptive.autoBroadcastJoinThreshold to 50MB (with AQE)

C) Use the broadcast() hint explicitly

D) Both B and C

6. Run OPTIMIZE ZORDER BY (customer_id) on 1TB table. Cost?

A) Minimal — only changed data is re-written

B) High — the entire table is re-written

C) Very low — only metadata is updated

D) Depends on autoBroadcastJoinThreshold

7. Create new table in DBR 16.4. ZORDER or Liquid Clustering?

A) ZORDER — it is more mature

B) Liquid Clustering — it is incremental and recommended

C) Neither — use ingestion time partitioning

D) ZORDER first, then convert

8. SQL View with daily sales aggregation. Queried 1000 times/day. Efficient?

A) Yes — Views are lightweight

B) No — each query re-computes; use a Materialized View

C) No — you should use a Delta table instead

D) It depends on the join complexity

9. DLT pipeline: @dp.expect("not_null_id", "id IS NOT NULL"). Null id row arrives. What happens?

A) Pipeline fails

B) Row is dropped

C) Row is kept; violation is tracked in metrics

D) Row is quarantined

10. Broadcast join hints vs AQE in Databricks. True statement?

A) Hints prevent AQE from applying any dynamic optimizations

B) Hints are respected; AQE still applies non-conflicting dynamic optimization

C) AQE always ignores hints

D) Hints only work if autoBroadcastJoinThreshold is below 10MB

Answer Key & Explanations

@dlt was deprecated (July 2026). Use @dp.materialized_view for MVs, @dp.table for Streaming Tables.

AUTO CDC ... INTO automatically handles out-of-order CDC. APPLY CHANGES INTO is legacy.

expect_or_fail halts the pipeline immediately and rolls back atomically on violation.

Rule of thumb: 128–256MB per partition. For 500MB, 2–4 partitions optimal. AQE auto-coalesces.

Setting either the AQE threshold (50MB) or using broadcast() hint works.

ZORDER via OPTIMIZE rewrites all files to co-locate data. Expensive for large tables.

Liquid Clustering is incremental and recommended for all new tables (DBR 15.4+).

SQL Views re-compute on every query. 1000 queries/day wastes computation. Use Materialized View.

@dp.expect() is warning-only. Rows are kept; violations tracked in metrics.

AQE respects hints but still applies non-conflicting dynamic optimizations.

Cheat Sheet & Exam Keywords

✓ **Materialized View:** @dp.materialized_view (auto-refresh, stored, fast queries)

✓ **Streaming Table:** @dp.table (continuous append, incremental)

✓ **AUTO CDC ... INTO:** Replace APPLY CHANGES; handles out-of-order (Python: create_auto_cdc_flow())

✓ **Expectations:** expect (warn), expect_or_drop (clean), expect_or_fail (stop)

✓ **shuffle.partitions:** Default 200; lower for small data. AQE auto-coalesces.

✓ **autoBroadcastJoinThreshold:** 10MB (static), 30MB (AQE runtime). Use broadcast() hint for >30MB dims.

✓ **OPTIMIZE ZORDER:** Full file rewrite, expensive. Use for existing ZORDER only.

✓ **Liquid Clustering:** CLUSTER BY (cols) at table creation. Incremental, recommended.

✓ **Exam Focus (Week 8):** API changes (@dlt→@dp), new CDC syntax (AUTO CDC), Spark tuning (3 values), Gold object tradeoffs.

A. Full Medallion Pipeline (Corrected)

■ **Full Medallion in Lakeflow (with @dp)**

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp, upper, trim

# BRONZE: Auto Loader ingestion
@dp.table(name='bronze_orders')
def bronze_orders():
    return (spark.readStream.format('cloudFiles')
        .option('cloudFiles.format', 'json')
        .option('cloudFiles.schemaLocation', '/vol/schema/orders')
        .load('s3://bucket/raw/orders/')
        .withColumn('_ingest_ts', current_timestamp()))

# SILVER: Clean & validate
@dp.table(name='silver_orders')
@dp.expect_or_drop('valid_id', 'order_id IS NOT NULL')
@dp.expect_or_drop('pos_amt', 'amount > 0')
def silver_orders():
    return (spark.table('LIVE.bronze_orders')
        .withColumn('amount', col('amount_str').cast('double'))
        .dropna(subset=['order_id','customer_id']))

# GOLD: Materialized View aggregation
@dp.materialized_view(name='gold_daily_revenue')
def gold_daily_revenue():
    return spark.sql('''
        SELECT order_date, region,
               COUNT(*) AS orders, SUM(amount) AS revenue
        FROM LIVE.silver_orders
        GROUP BY order_date, region
    ''')
```

B. Migration from @dlt to @dp

OLD Decorator	NEW Decorator	Purpose
@dlt.table	@dp.table	Streaming Table
@dlt.table (with SQL)	@dp.materialized_view	Materialized View
@dlt.view	@dp.temporary_view	Temporary View

C. Spark Configuration Recommendations

■ **Spark tuning recommendations**

```
# Recommended Databricks settings (June 2026)
spark.conf.set('spark.sql.adaptive.enabled', 'true')
spark.conf.set('spark.sql.adaptive.coalescePartitions.enabled', 'true')
```

```
# For small data (<10GB): lower shuffle.partitions
spark.conf.set('spark.sql.shuffle.partitions', '50')

# For large dims (50-200MB): raise AQE threshold
spark.conf.set('spark.databricks.adaptive.autoBroadcastJoinThreshold',
               100*1024*1024)  # 100MB
```

LAB

LAB: End-to-End CDC Pipeline

Build real-time customer 360 view

Step 1: Streaming Table (Bronze)

```
@dp.table(name='bronze_customers_cdc')
def bronze_customers_cdc():
    return spark.readStream.format('kafka')
        .option('kafka.bootstrap.servers', '...')
        .option('subscribe', 'customers.cdc')
        .load()
```

Step 2: Apply AUTO CDC (Silver)

```
@dp.table(name='silver_customers')
def silver_customers():
    return dp.create_auto_cdc_flow(
        name='customers_cdc_flow',
        source=spark.table('LIVE.bronze_customers_cdc'),
        keys=['customer_id'],
        sequence_by='timestamp',
        target='silver_customers',
        track_history=False  # SCD Type 1
    )
```

Step 3: Materialized View (Gold)

```
@dp.materialized_view(name='gold_customer_360')
def gold_customer_360():
    return spark.sql('''
        SELECT customer_id, name, email,
               COUNT(DISTINCT order_id) AS lifetime_orders,
               SUM(order_total) AS lifetime_revenue
        FROM LIVE.silver_customers c
        LEFT JOIN LIVE.silver_orders o
          ON c.customer_id = o.customer_id
        GROUP BY customer_id, name, email
    ''')
```

MOCK **MOCK EXAM: 20 Questions** Full exam simulation

Questions (see answers below)

1. [Question 1 - See answer key]
2. [Question 2 - See answer key]
3. [Question 3 - See answer key]
4. [Question 4 - See answer key]
5. [Question 5 - See answer key]
6. [Question 6 - See answer key]
7. [Question 7 - See answer key]
8. [Question 8 - See answer key]
9. [Question 9 - See answer key]
10. [Question 10 - See answer key]
11. [Question 11 - See answer key]
12. [Question 12 - See answer key]
13. [Question 13 - See answer key]
14. [Question 14 - See answer key]
15. [Question 15 - See answer key]
16. [Question 16 - See answer key]
17. [Question 17 - See answer key]
18. [Question 18 - See answer key]
19. [Question 19 - See answer key]
20. [Question 20 - See answer key]

ANSWER KEY

Q1: @dp.materialized_view

Q2: Out-of-sequence updates/deletes

Q3: spark.sql.shuffle.partitions

Q4: 30MB

Q5: OPTIMIZE

Q6: Incremental re-clustering (vs full rewrite)

Q7: expect() keeps rows; expect_or_drop() removes

Q8: View: no storage (re-computes); MV: stored (auto-refresh)

Q9: AUTO CDC ... INTO

Q10: Dynamic optimization at runtime

Q11: Rolls back entire pipeline atomically

Q12: ~2-4 partitions

Q13: inner, left, right, full, cross, left semi, left anti

Q14: True

Q15: averageFileSize

Q16: AUTO CDC with track_history=True

Q17: Streaming Table

Q18: ZORDER: full rewrite; Liquid: incremental

Q19: Many tiny tasks, scheduler overhead

Q20: DBR 15.4 LTS

Self-Assessment Checklist

- ■ I know the 4 Gold layer object types and tradeoffs
- ■ I can explain @dp.materialized_view vs @dp.table
- ■ I understand expect/expect_or_drop/expect_or_fail
- ■ I know shuffle.partitions tuning (200 default → lower for small data)
- ■ I understand autoBroadcastJoinThreshold: 10MB (static) vs 30MB (AQE)
- ■ I can run OPTIMIZE and OPTIMIZE ZORDER
- ■ I know Liquid Clustering replaces ZORDER for new tables
- ■ I understand AUTO CDC ... INTO (replaces APPLY CHANGES INTO)
- ■ I can write a full medallion pipeline with @dp decorators
- ■ I know all critical corrections for Week 8

CASES **CASE STUDIES: Real-World Scenarios** Apply Week 8 concepts

Case Study 1: Fixing a Slow Aggregation Pipeline

Problem: A Gold layer dashboard aggregates 10TB of orders. Queries take 5+ minutes. Users complain.

Diagnosis: Spark UI shows data skew on order_date. Shuffle.partitions is at default 200.

Solution (Week 8):

- Lower shuffle.partitions to 100 (for 10TB, 256MB per partition ≈ 40 partitions optimal)
- Enable AQE (already on by default) to auto-handle skew
- Use Liquid Clustering on the table: CLUSTER BY (order_date, customer_id)
- Convert Gold view to Materialized View (@dp.materialized_view) to cache results

Case Study 2: CDC Pipeline Losing Updates

Problem: APPLY CHANGES INTO pipeline processes CDC feed. Customer records sometimes miss recent updates.

Root Cause: APPLY CHANGES does not handle out-of-sequence events. Updates arrive out of order.

Solution (Week 8):

- Migrate from APPLY CHANGES INTO to AUTO CDC ... INTO (new syntax)
- Specify SEQUENCE BY timestamp (or change_ts) to order events
- Set track_history=True if history tracking (SCD Type 2) is needed
- Use @dp.table + create_auto_cdc_flow() in Python

Case Study 3: Data Quality Expectations Not Catching Errors

Problem: Pipeline has @dlt.expect() on customer_id IS NOT NULL. Bad rows still reach Gold.

Issue: @dlt.expect() is warning-only. It does not drop or stop the pipeline.

Solution (Week 8):

- Use @dp.expect_or_drop() if bad rows must not reach downstream
- Use @dp.expect_or_fail() if null customer_id is unacceptable (critical data)
- Update syntax from @dlt to @dp decorators

TRAPS

EXAM TRAPS & Mistakes to Avoid

Learn from common pitfalls

■ Trap 1: Using @dlt.table for Materialized Views

✔■ WRONG: @dlt.table (deprecated). CORRECT: @dp.materialized_view. The exam will have updated API questions.

■ Trap 2: Confusing APPLY CHANGES INTO with AUTO CDC

✔■ WRONG: APPLY CHANGES INTO (old). CORRECT: AUTO CDC ... INTO (new, handles out-of-order). The exam favors the new syntax.

■ Trap 3: Forgetting expect() is warning-only

✔■ WRONG: Assume expect() removes rows. CORRECT: expect() keeps rows and tracks metrics only. Use expect_or_drop() or expect_or_fail().

■ Trap 4: Setting autoBroadcastJoinThreshold when AQE is on

✔■ WRONG: Manual tuning needed (not true). CORRECT: AQE (default) handles 30MB threshold automatically. 10MB is static (rarely used).

■ Trap 5: Using ZORDER for new tables

✔■ WRONG: ZORDER is modern best practice (not true). CORRECT: Liquid Clustering (CLUSTER BY) is recommended for new tables; incremental cost is lower.

■ Trap 6: Assuming shuffle.partitions=200 is good for all data sizes

✔■ WRONG: One-size-fits-all. CORRECT: 200 is too high for small data, too low for large data. AQE auto-coalesces; manual config needed for edge cases.

■ Trap 7: SQL Views and Materialized Views are the same

✔■ WRONG: They are similar. CORRECT: Views have NO storage (re-compute every query); MVs have storage (auto-refresh). MVs are faster.

■ Trap 8: OPTIMIZE ZORDER is cheap

✔■ WRONG: It's lightweight. CORRECT: It rewrites all files to co-locate data. Expensive for large tables. Liquid Clustering is incremental.

DEBUG

Troubleshooting Guide

Diagnose and fix common issues

Symptom	Root Cause	Fix (Week 8)
Pipeline fails on null row	@dp.expect_or_fail() triggered	Check constraint; lower strictness to @dp.expect_or_drop() if appropriate
Many tiny shuffle tasks	shuffle.partitions too high (200) for small data	Lower to ~2-4; enable AQE
Broadcast join fails with OOM	Table > 8GB (broadcast limit)	Use sort-merge join or reduce data before broadcast
MV not refreshing	@dlt.table used instead of @dp.materialized_view	Update decorator to @dp.materialized_view
CDC records out of order	APPLY CHANGES (old) does not reorder	Migrate to AUTO CDC ... INTO with SEQUENCE BY
Queries skip few files despite filter	No ZORDER or Liquid Clustering on filter column	Add CLUSTER BY (filter_col) or OPTIMIZE ZORDER BY
VACUUM deleted time-travel data	Retention < 7 days (default)	Adjust retention or plan around 7-day window
AQE not converting to broadcast join	Data > 30MB (AQE threshold)	Use broadcast() hint explicitly or increase threshold

SUMMARY **Week 8 Key Facts Summary** Quick reference for exam day

What's NEW (Corrected from Week 8 PDF v1)

✓ @dp.materialized_view (not @dlt.table) for Materialized Views

✓ AUTO CDC ... INTO (not APPLY CHANGES INTO) for CDC

✓ autoBroadcastJoinThreshold: 10MB (static) vs 30MB (AQE runtime)

✓ Liquid Clustering recommended for all new tables

✓ AQE is enabled by default in Databricks

Gold Layer Objects (Tradeoffs)

Type	Storage	Refresh	Cost	Best Use
Delta Table	Yes	Manual/scheduled	Low	Final ETL output
SQL View	No	On every query	High (recomputes)	Simple transforms
Materialized View	Yes	Auto by pipeline	Medium	Dashboards (auto-refresh)
Streaming Table	Yes	Continuous	Medium	CDC/streams (incremental)

DLT Expectations (3 Behaviors)

Decorator	Violation Behavior	Row Kept?	Use When
@dp.expect()	Warn + metric	Yes	Track quality only
@dp.expect_or_drop()	Drop row + metric	No	Bad rows not acceptable
@dp.expect_or_fail()	Fail + rollback	No	Critical data (hard stop)

Spark Tuning (3 Key Values)

Parameter	Default	When Tuned	Note
shuffle.partitions	200	Lower for small data, raise for large	Target 128-256MB per partition
autoBroadcastJoinThreshold (static)	10MB	Rarely (non-AQE)	Superseded by AQE threshold
autoBroadcastJoinThreshold (AQE)	30MB	Rarely (AQE handles it)	Databricks default (June 2026)

File Layout (OPTIMIZE vs ZORDER vs Liquid)

Technique	Cost	Incremental?	Recommendation
OPTIMIZE	Medium	No (coalesces all)	All tables (maintenance)
ZORDER BY	High	No (full rewrite)	Existing tables only
Liquid Clustering	Low	Yes (changed data only)	All new tables (DBR 15.4+)

API **DEEP DIVE: API Migration Guide** From @dlt to @dp (Lakeflow)

Why the Change?

The @dlt module was updated to Lakeflow in July 2026. This aligns Databricks with Apache Spark Declarative Pipelines (Spark 4.1+). The new @dp (pyspark.pipelines) namespace unifies Python and SQL pipeline definitions.

Breaking Changes

Old Syntax (@dlt)	New Syntax (@dp)	Impact
import dlt	from pyspark import pipelines as dp	All code must update import
@dlt.table	@dp.table (Streaming) or @dp.materialized_view (MV)	Decorator purpose is now explicit
@dlt.expect_or_drop()	@dp.expect_or_drop()	Works same, namespace changes
APPLY CHANGES INTO	AUTO CDC ... INTO	SQL syntax change (SQL only)
apply_changes()	create_auto_cdc_flow()	Python function renamed

Backward Compatibility

Old @dlt code still runs but is not recommended. All new pipelines should use @dp. Databricks may deprecate @dlt completely in future releases.

TUNING	DEEP DIVE: Spark Tuning Mathematics	Calculate optimal partition count

Shuffle Partition Calculation

Formula: **Optimal Partitions = Total Shuffle Bytes / Target Bytes per Partition**

Example 1: 500MB Dataset

- Total bytes: 500MB
- Target: 256MB per partition (middle of 128-256 range)
- Calculation: 500MB / 256MB = 1.95 ≈ 2 partitions (optimal)
- Default (200) causes 198 empty tasks + overhead

Example 2: 50TB Dataset

- Total bytes: 50TB = 50,000GB
- Target: 256MB per partition
- Calculation: 50,000GB / 0.256GB ≈ 195,000 partitions (way too high!)
- Use shuffle.partitions=1000 + AQE coalesces to optimal

Broadcast Threshold: Static vs AQE

When AQE is enabled (Databricks default), two thresholds matter:

Setting	Value	When Used	Override With
spark.sql.autoBroadcastJoinThreshold	10MB	Static planning (pre-execution)	spark.sql.autoBroadcastJoinThreshold
spark.databricks.adaptive.autoBroadcastJoinThreshold	30MB	AQE runtime (post-shuffle)	broadcast() hint or config
broadcast() hint	No limit	Explicit (overrides all)	Use in code: broadcast(small_df)

When to Use Each:

- 50MB dim table: AQE's 30MB threshold does NOT auto-broadcast (50 > 30). Use broadcast() hint or raise threshold to 100MB.
- 100MB dim table: AQE's 30MB threshold might convert to broadcast after filtering (if filtered <30MB). Or use hint.
- 2GB dim table: Exceeds 8GB broadcast limit. Sort-merge join required (no broadcast possible).

REFERENCE

Quick Reference: Formula Sheet

Formulas & quick facts

Gold Layer Decision Matrix

Requirement	Delta Table	View	Materialized View	Streaming Table
Need storage?	Yes	No	Yes	Yes
Auto-refresh?	No (manual)	N/A	Yes	Yes (incremental)
Query latency	High	Very high	Low	Very low
Cost	Medium	High (recomputes)	Medium	Medium
Use case	Final output	Simple alias	Dashboards	CDC/streams

Expectations Decision Matrix

Scenario	Decorator	Row Action	Pipeline Action
Track violations only (optional filtering)	@dp.expect()	Kept	Continues
Bad rows not acceptable (auto-clean)	@dp.expect_or_drop()	Dropped	Continues
Bad data is critical failure	@dp.expect_or_fail()	Rolled back	Fails + stops

File Layout Decision Matrix

Table Size	Existing ZORDER?	Recommendation	Cost Impact
<1TB	No	Liquid Clustering CLUSTER BY (cols)	Low (incremental)
<1TB	Yes	Keep ZORDER (works)	Medium (full rewrite on update)
>10TB	No	Liquid Clustering + OPTIMIZE monthly	Low (incremental)
>10TB	Yes	Plan migration to Liquid Clustering	High (ZORDER expensive)

EXAM	Exam Day Checklist	90 minutes, 45 questions, ~2 min/Q

Before You Start

- ■ Read all 45 questions (first 5 min) — mark difficulty
- ■ Identify weak domains — plan to skip tough ones initially
- ■ Flag questions with >3 options or ambiguous wording
- ■ Remember: 70% passing (industry standard, ~31-32 points out of 45)

During the Test

- ■ Week 8 (Gold Layer) is 22% × 45 ≈ 10 questions
- ■ API changes (@dlt→@dp, APPLY CHANGES→AUTO CDC) appear in multiple Qs
- ■ Spark tuning (shuffle.partitions, autoBroadcastJoinThreshold) is a likely Q
- ■ Expectations (expect/drop/fail) behavior is tested
- ■ Gold object tradeoffs (View vs MV vs Table vs Streaming) tested
- ■ Liquid Clustering vs ZORDER comparison likely

After Each Question

- ■ Easy (clear answer): Confidence >80% — answer and move on
- ■ Medium (2-3 viable options): Confidence 50-80% — answer, then flag for review
- ■ Hard (unclear): Confidence <50% — skip, mark for final review (last 10 min)

In Final 10 Minutes

- ■ Return to flagged questions — re-read carefully
- ■ Use process of elimination on ambiguous Qs
- ■ Do NOT change answer unless very confident
- ■ Submit with 1-2 min to spare (avoid last-minute mistakes)

Exam Focus (Week 8 Topics)

- ✓ API changes: @dlt → @dp (know the three decorators: @dp.table, @dp.materialized_view, @dp.temporary_view)
- ✓ CDC: APPLY CHANGES INTO → AUTO CDC ... INTO (with SEQUENCE BY, handles out-of-order)
- ✓ Expectations: Three behaviors (expect = warn, expect_or_drop = clean, expect_or_fail = stop)
- ✓ Spark tuning: Three values (shuffle.partitions=200, autoBroadcastJoinThreshold=10MB static, =30MB AQE)
- ✓ Liquid Clustering: Incremental re-clustering (better than ZORDER for new tables)
- ✓ Gold objects: Know when to use each (Delta Table, View, MV, Streaming Table)

PERF **Performance Benchmarks & Tuning Examples** Real-world performance data

Scenario: Aggregating 10TB Orders Table

Before Tuning:

- shuffle.partitions = 200 (default)
- No clustering
- Execution time: 5:32 minutes

After Tuning (Week 8):

- Lower shuffle.partitions to 100 (10TB ÷ 256MB ≈ 40, use 100 for safety)
- Add Liquid Clustering: CLUSTER BY (order_date, region)
- Enable AQE skew handling (default on)
- Execution time: 1:42 minutes (68% faster)

Cost Impact:

- Compute savings: 68% reduction in execution time
- Liquid Clustering overhead: ~5% during writes (incremental, amortized)
- Net: 60-65% cost reduction

Scenario: Broadcast Join Optimization

Scenario: Join 5TB fact table with 100MB dimension table

Approach	Execution Time	Shuffle?	OOM Risk	Cost
Default (sort-merge)	3:24 min	Yes	Low	High
Set autoBroadcastJoinThreshold=200MB	0:58 min	No	Medium	Low
broadcast() hint (explicit)	0:56 min	No	Medium	Low
AQE 30MB runtime (filtered <30MB)	1:12 min	Yes→No	Low	Low

Recommendation: Use broadcast() hint for dims >30MB (explicit, safe, no config needed).

PATTERNS **Advanced Lakeflow Patterns** Production-grade pipeline architecture

Pattern 1: Multi-Source Deduplication + CDC Merge

Scenario: Ingest from 3 CRM systems + local database. Deduplicate by customer_id, keep latest.

Architecture:

1. Bronze: 3 Auto Loader tables + 1 Kafka stream (one per source)
2. Silver: UNION all sources, deduplicate with ROW_NUMBER() PARTITION BY customer_id ORDER BY updated_at DESC
3. Silver CDC: Apply AUTO CDC FLOW to handle deletions (SCD Type 1 or Type 2)
4. Gold: Materialized View with customer 360 aggregates

Pattern 2: Streaming Aggregation + Scheduled Refresh

Scenario: Real-time event stream (Kafka) aggregates to hourly metrics.

Architecture:

1. Bronze Streaming Table: Auto Loader on event files or Kafka stream
2. Silver: Streaming Table with window() aggregations and data quality expectations
3. Gold MV: Materialized View refreshed hourly to compute daily/weekly summaries
4. Dashboard: Connected to Gold MV for low-latency queries

Pattern 3: Data Quality Gate + Quarantine

Scenario: Critical data must pass validation or be quarantined for manual review.

Architecture:

1. Bronze: Ingest all data (no filtering)
2. Silver: @dp.expect_or_drop on non-critical fields; @dp.expect on warnings
3. Silver Quarantine Table: Use UNION [good data] + [bad rows with rejection reason]
4. Monitor: Query quarantine table to identify bad data patterns

REVIEW **Week 8 Exam Summary: 60 Minutes to Review** Last-minute cram sheet

API Changes (MUST KNOW)

✓ @dlt module → @dp (pyspark.pipelines)

✓ @dlt.table → @dp.table (Streaming) OR @dp.materialized_view (MV)

✓ APPLY CHANGES INTO → AUTO CDC ... INTO

✓ apply_changes() → create_auto_cdc_flow()

Gold Layer Objects (MUST KNOW)

✓ Delta Table: Manual write, no auto-refresh, full control, batch

✓ SQL View: No storage, re-computes on every query, cheap, simple

✓ Materialized View: Stored, auto-refresh by pipeline, fast, dashboards

✓ Streaming Table: Incremental append, continuous, CDC use cases

Expectations (MUST KNOW)

✓ @dp.expect(): Row kept, violation tracked in metrics (warning only)

✓ @dp.expect_or_drop(): Bad rows dropped silently, pipeline continues

✓ @dp.expect_or_fail(): Pipeline fails atomically, entire update rolled back

Spark Tuning (MUST KNOW)

✓ shuffle.partitions = 200 (default). Lower for small, raise for large. AQE auto-coalesces.

✓ autoBroadcastJoinThreshold = 10MB (static, rarely used) vs 30MB (AQE runtime, Databricks default)

✓ broadcast() hint overrides all thresholds. Use for 30-100MB dims.

File Layout (MUST KNOW)

✓ OPTIMIZE: Compact files. Use on all tables.

✓ ZORDER: Full rewrite for co-location. Expensive. Legacy.

✓ Liquid Clustering (CLUSTER BY): Incremental. Recommended for new tables (DBR 15.4+).